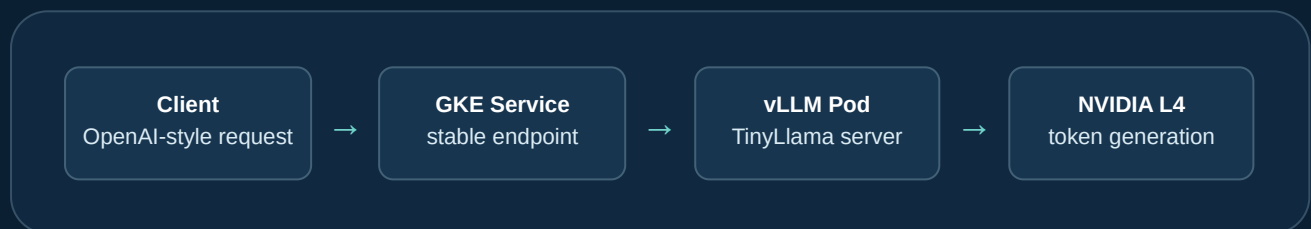


LLM Serving on GKE

A technical build diary, architecture explanation, troubleshooting record, and interview-preparation guide for GPU inference with vLLM on Google Kubernetes Engine.



Project state, 28 September 2026: inference verified, two production-style failures diagnosed, Horizontal Pod Autoscaler exercised under load, GPU capacity ceiling observed, and the workload parked at zero replicas to control cost.

The project proves operating judgment, not just deployment syntax

A model answering one prompt is a demo. A platform engineer must also explain scheduling, rollout behavior, metrics, failure modes, capacity constraints, and cost. This project produced evidence in each category.

60

completion tokens returned in the verified inference test

11m → 171m

pod CPU from idle to six-client load

1 → 2

replicas requested when HPA target was lowered

WHAT WAS BUILT

- A Google Kubernetes Engine (GKE) cluster with a GPU node pool capable of scaling from zero.
- A Kubernetes Deployment running the official `vllm/vllm-openai` image with TinyLlama 1.1B on one NVIDIA L4 GPU.
- A ClusterIP Service on port 8000, accessed safely for development through `kubectl port-forward`.
- A CPU-based Horizontal Pod Autoscaler (HPA), then a controlled test showing where that signal and the available capacity fail.

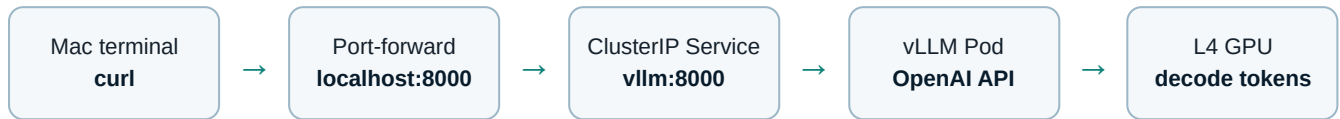
WHAT MADE THE PROJECT INTERVIEW-WORTHY

- **Root-cause analysis:** fixed a vLLM environment collision by setting `enableServiceLinks: false`.
- **Controller reasoning:** changed Deployment strategy to `Recreate` after a one-GPU RollingUpdate deadlock.
- **Auditability:** proved that Kubernetes' pod garbage collector, not a human, deleted finished orphaned pods after a GPU node scaled to zero.
- **Evidence-driven autoscaling:** observed that CPU utilization stayed low despite meaningful inference traffic, motivating queue-based custom metrics.

Interview thesis: “I built a small but complete inference platform and intentionally pushed it into failure states. I used events, logs, audit records, resource metrics, and controller behavior to explain exactly why it behaved as it did.”

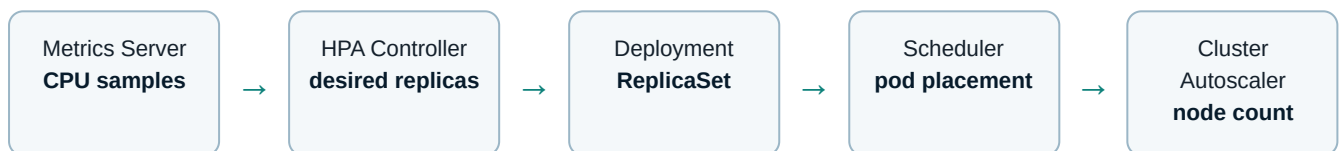
Two planes work together: serving traffic and reconciling desired state

DATA PLANE: HOW ONE INFERENCE REQUEST MOVES



The Service selects pods labeled `app: vllm`. Port-forward is a development tunnel, not a production ingress. The request targets `/v1/chat/completions`, vLLM schedules model work on the GPU, and the response returns usage metadata alongside generated text.

CONTROL PLANE: HOW KUBERNETES REACTS



These are separate feedback loops. HPA changes the number of pods. The scheduler tries to place those pods. Only when a pod is unschedulable for capacity reasons can the cluster autoscaler decide to add a node. A quota or node-pool maximum can stop that chain.

CURRENT IMPLEMENTATION CHOICES

Layer	Choice	Reason
Model server	vLLM OpenAI API	Standard HTTP contract; GPU-oriented serving
Model	TinyLlama 1.1B Chat	Small enough for an inexpensive learning deployment
Exposure	ClusterIP + port-forward	Avoids public endpoint while developing
GPU allocation	1 GPU per pod	Explicit extended resource request
Rollout	Recreate	Only one GPU available
Idle state	Deployment scaled to 0	Allows GPU node pool to drain and reduce cost

The sequence moved from “can it run?” to “can it explain itself?”

- 1 **Prepared GCP and GKE.** Confirmed billing, created a cluster, and used a separate GPU node pool with autoscaling. The pool could drain when no GPU pod required it.
- 2 **Deployed vLLM.** Ran TinyLlama/TinyLlama-1.1B-Chat-v1.0 from the official image, requested one GPU, set `--max-model-len 2048`, and exposed port 8000 through a Service.
- 3 **Diagnosed startup failure.** Identified a Kubernetes Service-link environment variable collision with vLLM configuration; disabled automatic Service environment injection.
- 4 **Diagnosed rollout deadlock.** A RollingUpdate attempted to start a new GPU pod before terminating the old one. With one GPU, the new pod could not schedule. Switching to Recreate released the GPU first.
- 5 **Verified serving.** Waited for `Application startup complete`, checked `/health`, and sent an OpenAI-compatible chat request. The server returned 60 completion tokens.
- 6 **Investigated pod deletion.** Cloud Audit Logs named `system:serviceaccount:kube-system:pod-garbage-collector` as the deleter after the GPU node scaled down. This replaced speculation with evidence.
- 7 **Enabled HPA math.** Added a 1 CPU request, created HPA at 70% with min 1 and max 3, and confirmed Metrics Server supplied a real utilization value.
- 8 **Load-tested and forced scale-out.** Six concurrent loops raised CPU from 11m to 171m, but HPA reached only 22%. Lowering the target to 10% produced a second pod, which remained Pending because GPU capacity was exhausted.
- 9 **Parked safely.** Deleted HPA before scaling the Deployment to zero, then stopped local load and port-forward processes. This avoided a controller recreating the pod.

Engineering pattern: build one layer, verify its observable output, introduce controlled pressure, explain the resulting state, then clean up.

Each Kubernetes object owns a different part of the outcome

DEPLOYMENT

Stores the desired pod template and replica count. It manages rollout behavior and creates ReplicaSets. In this project it also carries the Recreate strategy.

REPLICASET

Maintains the requested number of matching pods for a specific template revision. The Deployment normally manages it; operators rarely edit it directly.

POD

The schedulable unit. It contains the vLLM container, resource requests, limits, labels, and runtime state. Pods are replaceable, not durable pets.

SERVICE

Provides stable discovery and load-balancing across matching pod endpoints. A ClusterIP is internal to the cluster.

NODE AND NODE POOL

The node supplies CPU, memory, and an attached L4. The node pool defines homogeneous capacity and autoscaling bounds.

CONTROLLERS

Deployment controller, HPA, garbage collector, and cluster autoscaler continuously compare observed state with their goals and act to reduce differences.

DECLARATIVE VERSUS IMPERATIVE CHANGES

Declarative: `kubectl apply -f vllm.yaml` says “make the cluster match this configuration.”

Imperative: `kubectl scale deploy vllm --replicas=0` directly changes a live field. Mixing ownership of the same field creates surprises.

The observed trap: the manifest and last-applied configuration both said `replicas: 1`, while an imperative scale changed the live object to 0. Re-applying an otherwise unchanged manifest may not be the clearest or safest way to “undo” that manual scale. The practical rule used here was: if replica count was changed with `scale`, restore it with `scale`; let the manifest own the stable configuration afterward.

INTERVIEW DISTINCTION

Kubernetes is not executing a one-time script. It is a collection of reconciliation loops. A command changes desired state; controllers then create, delete, or update lower-level objects until observed state approaches it.

A GPU request is a hard placement constraint

Kubernetes advertises GPUs as extended resources such as `nvidia.com/gpu`. They are discrete: a pod requesting one GPU needs a node with one allocatable GPU.

```
resources:
  requests:
    cpu: "1"
  limits:
    nvidia.com/gpu: 1
```

WHAT HAPPENS AT SCHEDULING TIME

1. The scheduler filters nodes that cannot satisfy CPU, memory, taints/tolerations, selectors, affinity, and extended-resource requirements.
2. If no feasible node exists, the pod remains Pending; this is not a container or application failure because the container has not started.
3. The cluster autoscaler evaluates whether expanding a matching node pool would make the pod schedulable.
4. If the pool is already at its maximum, the event records `NotTriggerScaleUp`.

WHAT WAS OBSERVED

```
Warning FailedScheduling 0/2 nodes are available:
1 Insufficient cpu, 2 Insufficient nvidia.com/gpu.

Normal NotTriggerScaleUp Pod didn't trigger scale-up:
1 max node group size reached
```

This output separates two layers. The scheduler explained why current nodes could not host the pod. The cluster autoscaler explained why it would not add capacity.

QUOTA

Cloud quota determines whether the project may create another accelerator in the region or zone.

NODE-POOL MAXIMUM

The autoscaler cannot exceed the configured maximum, even when a workload is unschedulable.

HPA MAXIMUM

Controls desired pod replicas, not available GPUs. A higher value does not create physical capacity.

COLD START

Scale-from-zero requires VM provisioning, driver readiness, image availability, model loading, and application warmup.

Interview answer: "Pending means start with scheduling events, not application logs. If the pod has no node, there is no container log to inspect yet."

Serving was verified at three layers, not assumed from pod status

vLLM provides an OpenAI-compatible HTTP server and an official `vllm/vllm-openai` container image.¹ The project launched the TinyLlama chat model with a maximum model length of 2048 tokens and exposed container port 8000.

LAYER 1: KUBERNETES STATE

```
kubectl get pods
NAME          READY   STATUS    RESTARTS
vllm-...      1/1     Running   0
```

Running means the container process exists. It does not guarantee the model has finished loading or the server is ready to answer.

LAYER 2: APPLICATION READINESS

```
kubectl logs deploy/vllm | grep "Application startup complete"
(APIServer pid=1) INFO: Application startup complete.
```

The startup line proved that vLLM completed initialization. A health request through the Service tunnel then proved network reachability.

LAYER 3: FUNCTIONAL INFERENCE

```
curl -s http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"model": "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
  "messages": [{"role": "user", "content": "Why is the sky blue?"}],
  "max_tokens": 60}'
```

The response contained model-generated text and usage data: 21 prompt tokens, 60 completion tokens, and 81 total tokens. This was the end-to-end proof: request format, Service routing, model execution, GPU access, and response serialization all worked together.

Why three layers matter: “Pod Running,” “server initialized,” and “correct request returns tokens” are progressively stronger claims. A good verification plan names the layer each check proves.

CURRENT LIMITATIONS TO SAY PLAINLY

- The endpoint is development-only through port-forward, not a production ingress.
- The image uses the mutable `latest` tag and should be pinned.
- No durable model cache or production authentication layer has been added yet.

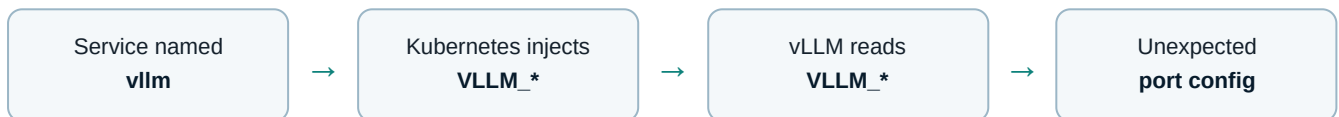
A platform convenience silently became application configuration

SYMPTOM

vLLM failed during startup with port-related behavior that did not match the explicit container arguments. The Deployment and Service looked reasonable, so the investigation moved to the container environment.

MECHANISM

Kubernetes can inject environment variables describing Services visible to a pod. A Service named `vllm` can create variables with the `VLLM_` prefix. vLLM itself also consumes environment variables with that prefix. The namespace collision allowed infrastructure-generated values to be interpreted as application settings.



FIX

```
spec:
  template:
    spec:
      enableServiceLinks: false
      containers:
      - name: vllm
        image: vllm/vllm-openai:latest
```

Disabling Service links removed the automatically generated environment variables. DNS-based Service discovery still worked, which is the preferred mechanism for normal in-cluster access.

WHY THIS IS A STRONG INTERVIEW STORY

- **It crosses layers:** Kubernetes networking metadata affected application startup.
- **It avoids cargo-cult fixes:** the change is tied to a concrete namespace collision.
- **It suggests prevention:** inspect env, compare application prefixes with generated variables, and prefer explicit configuration.

General lesson: defaults are part of the runtime contract. When an application behaves as though it received an argument you never supplied, inspect environment variables, mounted configuration, admission mutations, and image defaults.

FOLLOW-UP HARDENING

Pin the vLLM image version, record the effective command and environment, and add a smoke test that starts the server and checks `/health` before promoting a new manifest.

RollingUpdate required temporary capacity the cluster did not have

DEFAULT BEHAVIOR

A Deployment normally performs a RollingUpdate: it creates replacement pods while old pods still run, then shifts toward the new ReplicaSet. This protects availability when spare capacity exists.

WHY IT STALLED HERE



The replacement pod stayed Pending because the old pod still occupied the only L4. The rollout could not progress until capacity was released, but the strategy was intentionally keeping the old pod alive.

PROJECT DECISION

```
spec:
  replicas: 1
  strategy:
    type: Recreate
```

Recreate terminates the existing pod before starting the new revision. That creates downtime, but it matches a learning environment with one scarce GPU.

Strategy	Benefit	Trade-off
RollingUpdate	Maintains service	Needs surge capacity
Recreate	Works with one GPU	Deliberate downtime
Blue/green	Fast rollback	Doubles capacity during cutover

PRODUCTION INTERPRETATION

Recreate is not “better”; it is correct for the current constraint. Production choices could include at least two GPU replicas, a surge-capable pool, a planned maintenance window, or routing-aware rollout orchestration. The key is to make rollout capacity explicit.

Interview answer: “I changed strategy because the default encoded an availability assumption my cluster could not satisfy. I accepted downtime in the lab rather than leaving a rollout permanently blocked.”

Audit logs named the actor and replaced guesswork with a lifecycle explanation

OBSERVATION

A finished pod disappeared after the GPU node pool scaled to zero. From the terminal alone, this could look like a manual deletion, a Deployment action, or an autoscaler side effect.

EVIDENCE

Cloud Audit Logs recorded the deleting principal as:

```
system:serviceaccount:kube-system:pod-garbage-collector
```

This showed that a Kubernetes system controller deleted the object. The pod had become orphaned after its node disappeared during scale-down; the garbage collector cleaned up the stale object.

WHY AUDIT LOGS WERE DECISIVE

EVENTS

Best for recent scheduling, image pull, probe, and autoscaling explanations. They are local and time-limited.

CONTAINER LOGS

Best for application behavior after a container starts. They do not prove who changed a Kubernetes object.

AUDIT LOGS

Record control-plane API actions, including the authenticated principal and operation.

CONTROLLER STATE

Explains desired versus observed resources, ownership references, and reconciliation.

REUSABLE INVESTIGATION SEQUENCE

1. Confirm the object and namespace; capture its UID if available.
2. Check `kubectl get events --sort-by=.lastTimestamp`.
3. Inspect owner references and controller status.
4. Query Kubernetes API audit logs for delete operations and identify the principal.
5. Correlate the timestamp with node deletion or autoscaler activity.

Interview signal: avoid saying “Kubernetes deleted it” without evidence. Name the controller identity, the API action, and the preceding state transition.

RELIABILITY IMPLICATION

Pods must be treated as ephemeral. Durable artifacts such as model caches, logs, or request state need storage or external systems whose lifecycle is not tied to one node.

The CPU request is both a scheduling promise and an HPA denominator

CPU REQUEST

A reservation used by the scheduler. A request of `cpu: "1"` means 1000 millicores, or one CPU core of requested capacity.

CPU LIMIT

A runtime ceiling. Exceeding it can cause CPU throttling. The project intentionally set no CPU limit so vLLM could burst.

GPU LIMIT

`nvidia.com/gpu: 1` allocates one discrete accelerator. GPU resources are not overcommitted like CPU time.

QOS CLASS

Because not every resource had equal requests and limits, the pod appeared as `Burstable`.

THE UTILIZATION CALCULATION

$$\text{CPU utilization \%} = \text{current CPU usage} / \text{requested CPU} \times 100$$

Observed usage	Request	Utilization
11m	1000m	about 1.1%
171m	1000m	about 17.1%
HPA sample	1000m	22% at one point

Kubernetes documents resource utilization as usage divided by the requested resource.² Without a CPU request, a utilization-based HPA lacks a denominator for that pod and can report the metric as unknown.

HOW HPA CHOOSES REPLICAS

$$\text{desired replicas} = \text{ceil}(\text{current replicas} \times \text{current metric} / \text{target metric})$$

With one pod, 18% current utilization, and a 10% target: $\text{ceil}(1 \times 18/10) = 2$. That matches the observed HPA result. At 22% with a 70% target: $\text{ceil}(1 \times 22/70) = 1$, so it stayed at one replica.

Nuance: HPA averages across eligible pods, applies tolerance and stabilization behavior, and may account for missing or not-yet-ready metrics. The formula is the core intuition, not the complete controller implementation.

Six clients increased work, but CPU still failed as a saturation signal

EXPERIMENT DESIGN

Six background loops repeatedly sent chat-completion requests through a local port-forward. After allowing metrics to update, the pod and HPA were inspected with `kubectl top pod` and `kubectl get hpa`.

```
for i in 1 2 3 4 5 6; do
  while true; do
    curl -s http://localhost:8000/v1/chat/completions \
      -H "Content-Type: application/json" \
      -d '{"...": "max_tokens": 60}' > /dev/null
  done &
done
```

6

parallel request loops

171m

CPU after load, versus 11m idle

22%

HPA reading versus 70% target

INTERPRETATION

The experiment proved requests were arriving because CPU usage increased materially. However, the CPU target remained far away. In GPU inference, the accelerator performs most tensor computation; CPU may handle request parsing, tokenization, scheduling, and response handling without becoming the bottleneck.

Therefore, low CPU does not mean the serving system has spare inference capacity. It may still have a growing request queue, saturated GPU execution, rising time-to-first-token, or high key-value cache occupancy.

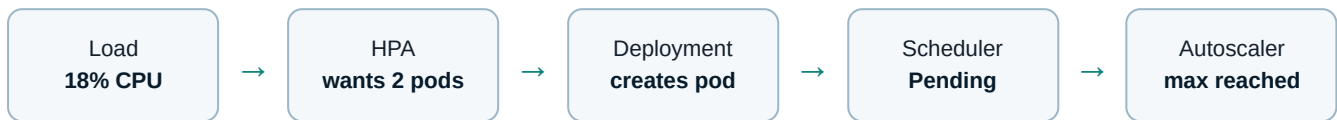
A USEFUL FAILURE DURING THE TEST

An older port-forward still owned local port 8000 but referenced a deleted pod sandbox. The new port-forward failed with `address already in use`, while traffic hit a dead tunnel. The recovery was to stop background jobs, terminate stale port-forward processes, confirm the port was free, establish a fresh tunnel, and health-check it before starting load.

Test-design lesson: a load generator must fail closed. Start load only after the endpoint passes health, record successful response counts or status codes, and always define a cleanup command. Otherwise, a busy client can create the illusion of server load.

Lowering the threshold proved every controller until the physical limit

To make the control path visible, the HPA target was temporarily lowered from 70% to 10%. With load still active, HPA reported 18%/10% and requested two replicas.



OBSERVED STATE

NAME	TARGETS	MINPODS	MAXPODS	REPLICAS
vllm	cpu: 18%/10%	1	3	2
vllm-...-mcwfg	1/1	Running		
vllm-...-w7xzp	0/1	Pending		

THREE CEILINGS, IN ORDER

1. **Cloud quota:** the provider must allow another L4 in the relevant location.
2. **Node-pool maximum:** GKE cluster autoscaler cannot grow beyond its configured limit.³
3. **HPA maximum:** HPA will never request more than `maxReplicas`, but that does not guarantee those replicas are schedulable.

The project configured one GPU of practical capacity and a node pool maximum of one. The event `max node group size reached` was therefore expected and safe: the second pod demonstrated the chain without provisioning another billable GPU.

WHY THIS MATTERS IN PRODUCTION

- Set HPA maximums from traffic needs, but set quotas and node-pool bounds from capacity planning.
- Monitor Pending pods and unschedulable duration; replica count alone can falsely suggest capacity exists.
- Include cold-start time in service-level objectives. GPU VM provisioning plus image and model loading can be minutes.
- Consider warm capacity, smaller images, model caching, or event-driven approaches when latency after scale-from-zero matters.

Core principle: an autoscaler is only as elastic as the scarcest upstream resource it is permitted to acquire.

Start with the layer that owns the symptom

Question	Command	What it proves
What exists?	<code>kubectl get pods -o wide</code>	State, node, age, restarts
Why Pending?	<code>kubectl describe pod NAME</code>	Scheduler and autoscaler events
Why restarting?	<code>kubectl logs NAME --previous</code>	Prior container failure
Did rollout finish?	<code>kubectl rollout status deploy/vllm</code>	Deployment progress
Is metrics live?	<code>kubectl top pod</code>	Resource-metrics pipeline
What does HPA think?	<code>kubectl describe hpa vllm</code>	Metrics, conditions, recommendations
Who deleted it?	Cloud Audit Logs	API action and principal
Who owns port 8000?	<code>lsof -i :8000</code>	Local process binding

DECISION TREE

- 1 Pending:** inspect Events. Think scheduler, capacity, selectors, taints, volumes, or quota.
- 2 ContainerCreating:** inspect Events. Think image pull, volume mount, sandbox, driver, or network setup.
- 3 Running but not Ready:** inspect logs and probes. Think application initialization or dependency reachability.
- 4 Ready but request fails:** test Service endpoints, port-forward, health route, request schema, and model name.
- 5 Object disappears:** inspect owner references, events, audit logs, and node lifecycle.

STATE CAPTURE BEFORE MUTATION

Before changing anything, save `get`, `describe`, recent events, and relevant logs. A successful fix can erase the failure state that would have explained the root cause.

Anti-pattern: repeatedly deleting pods until one works. That destroys evidence, may increase cost, and teaches nothing about the failure.

Observability will turn vLLM internals into operational signals

vLLM exposes Prometheus-format metrics at `/metrics`, including running and waiting requests and cache utilization.⁴ The next session will collect these metrics with Google Cloud Managed Service for Prometheus and visualize them in Cloud Monitoring.

PLANNED IMPLEMENTATION SEQUENCE

1. **Restart the workload safely.** Scale the Deployment from 0 to 1 and wait for the GPU node, container, model, and readiness.
2. **Verify the source.** Port-forward and inspect `/metrics`; confirm the exact metric names and labels emitted by the installed vLLM version.
3. **Enable managed collection.** Verify Managed Service for Prometheus is enabled for the GKE cluster.
4. **Create a PodMonitoring resource.** Select app: `vllm`, scrape the named/numbered port, and use path `/metrics`. Google documents PodMonitoring as the managed-collection resource for application exporters.⁵
5. **Confirm ingestion.** Query a known vLLM metric in Metrics Explorer or with PromQL before building a dashboard.
6. **Build a dashboard.** Put demand, saturation, latency, throughput, and errors on separate, readable panels.
7. **Run a controlled load.** Observe queue depth, concurrent running requests, cache usage, token throughput, and latency together.
8. **Add alerts.** Alert on sustained user-impact signals, not every transient fluctuation.

DASHBOARD PLAN

Panel	Signal	Question answered
Demand	request rate	How much traffic arrives?
Queue	requests waiting	Is demand exceeding service?
Concurrency	requests running	How full is execution?
Latency	TTFT / inter-token	What does the user feel?
Throughput	tokens per second	What work is completed?
Cache	KV cache usage	Is memory pressure rising?
Platform	CPU, memory, GPU	Which resource is saturated?

Verification gate: a dashboard is not complete until a known load causes a predictable visible change and the metric's labels are understood.

Queue depth should drive scale-out more directly than CPU

The load test showed CPU did not track inference pressure closely enough. vLLM publishes `vllm:num_requests_waiting` for queued requests and `vllm:num_requests_running` for work executing on the GPU.⁴ These metrics describe demand against serving capacity more directly.

TARGET CONTROL LOOP



DESIGN QUESTIONS TO RESOLVE EXPERIMENTALLY

- **Metric type:** per-pod waiting requests, total queue depth, request rate, or a composite signal.
- **Target:** a queue target low enough to protect latency, but high enough to avoid scaling on momentary bursts.
- **Scale-up speed:** GPU cold starts are slow, so aggressive scale-up may be appropriate once pressure is sustained.
- **Scale-down stability:** use a stabilization window so replicas are not removed during brief lulls.
- **Readiness:** new pods must not receive traffic before model initialization completes.
- **Capacity:** custom metrics cannot overcome quota or node-pool maxima; capacity must be increased deliberately.

CANDIDATE POLICY, TO VALIDATE RATHER THAN COPY BLINDLY

```

behavior:
  scaleUp:
    stabilizationWindowSeconds: 0
    policies:
      - type: Pods
        value: 1
        periodSeconds: 60
  scaleDown:
    stabilizationWindowSeconds: 300
metrics:
- type: Pods
  pods:
    metric:
      name: vllm_requests_waiting
    target:
      type: AverageValue
      averageValue: "1"
  
```

This is a design sketch, not completed configuration. The exact exposed metric name, adapter mapping, label aggregation, and target must be verified against the installed GKE and vLLM versions. Google has documented both adapter-based custom metrics and newer native Prometheus autoscaling paths; availability depends on cluster version and feature status.⁶

The path from lab to production is explicit and testable

RELEASE INTEGRITY

- Pin a tested vLLM image digest or immutable version.
- Store manifests in Git and review diffs.
- Add CI validation and a smoke-test deployment.
- Define rollback criteria and preserve prior revisions.

HEALTH AND LIFECYCLE

- Add startup, readiness, and liveness probes with different purposes.
- Give startup enough time for model download and warmup.
- Use a graceful termination period so in-flight requests can finish.
- Add a PodDisruptionBudget only when more than one replica exists.

MODEL DELIVERY

- Cache model weights on persistent or node-local storage where appropriate.
- Control model version and checksum.
- Measure image pull, weight download, load, and warmup separately.
- Plan Hugging Face access without embedding tokens in manifests.

SECURITY

- Keep the Service private unless public access is required.
- Use authenticated ingress/API gateway and TLS for external traffic.
- Apply least-privilege IAM and Workload Identity.
- Use Kubernetes Secrets or a secret manager; never plaintext in Git.
- Add NetworkPolicy and restrict control-plane access.

RELIABILITY

- Run multiple replicas across nodes/zones when the SLO requires it.
- Test node drain, pod eviction, image failure, and unavailable quota.
- Measure cold-start recovery and set error budgets.
- Alert on queue, latency, errors, and unschedulable pods.

PERFORMANCE

- Benchmark concurrency, prompt length, output length, batch behavior, TTFT, and tokens/second.
- Tune `max-model-len`, GPU memory utilization, and batching from evidence.
- Compare model size and quantization against quality requirements.

COST

- Keep GPU pools at zero when idle in the lab.
- Tag resources, watch budget alerts, and measure cost per 1K tokens.
- Evaluate spot capacity only for fault-tolerant workloads.
- Balance warm capacity against cold-start latency.

Interview framing: distinguish what was implemented from what is planned. Credibility increases when the roadmap is concrete and the current limitations are explicit.

Model answers connect mechanism, evidence, and trade-off

WHY DID THE SECOND POD REMAIN PENDING?

Answer: Each pod requested one `nvidia.com/gpu` resource. The running pod held the only L4. Scheduler events reported insufficient GPU, and the cluster autoscaler reported that the node pool had reached its maximum. HPA successfully requested capacity; the infrastructure could not supply it.

WHY DID HPA INITIALLY SHOW <UNKNOWN>?

Answer: Immediately after creation, the controller had not yet received a metric sample. That resolved to 1%. A different persistent unknown would occur if utilization-based HPA had no CPU request to use as its denominator.

WHY NO CPU LIMIT?

Answer: The project needed a scheduling request and HPA denominator, so it requested one CPU. A hard CPU limit could throttle short bursts. For this lab, unrestricted CPU bursting was preferable, while the GPU remained the hard constrained resource.

WHY USE RECREATE INSTEAD OF ROLLINGUPDATE?

Answer: RollingUpdate temporarily needed two GPUs: one for the old pod and one for the replacement. With only one, the rollout deadlocked. Recreate traded availability for feasibility. In production I would provision surge capacity or use a rollout design matched to the SLO.

WHY WAS CPU A POOR AUTOSCALING METRIC?

Answer: Six concurrent clients raised CPU from 11m to 171m, but HPA remained around 22% against a 70% target. The GPU was doing the model computation. Queue depth, concurrent requests, and latency are closer to the actual bottleneck.

HOW DID YOU PROVE WHO DELETED THE POD?

Answer: I used audit logs rather than inference. The recorded principal was the kube-system pod garbage collector. Correlating that with node scale-down explained the pod as an orphan cleanup.

HPA VERSUS CLUSTER AUTOSCALER?

Answer: HPA changes pod replicas based on metrics. The cluster autoscaler changes node count when pods cannot schedule for capacity reasons. They are separate loops connected by Pending pods.

The strongest story is a failure investigated to evidence

STAR ANSWER: SINGLE-GPU ROLLOUT

Situation: I was deploying an OpenAI-compatible vLLM server on GKE using a GPU node pool with one NVIDIA L4. An update created a new pod that remained Pending while the old pod continued running.

Task: Restore a repeatable rollout without increasing paid GPU capacity, while understanding the controller behavior rather than manually deleting resources.

Action: I inspected pod events and Deployment state. The replacement requested a GPU, but the existing replica still held the only device. The default RollingUpdate strategy assumed surge capacity that did not exist. I changed the Deployment to `strategy.type: Recreate`, accepting a brief outage in this lab. I then watched rollout status, checked the application startup log, health-tested the Service, and sent a real chat-completion request.

Result: The rollout completed reliably on one GPU and returned a 60-token completion. More importantly, I documented the trade-off: Recreate is appropriate for the current constraint, while a production service would need spare GPU capacity or a different deployment design to preserve availability.

RESUME BULLETS - ACCURATE TODAY

- Deployed an OpenAI-compatible vLLM inference service on Google Kubernetes Engine using an NVIDIA L4 GPU, Kubernetes Deployment/Service resources, and a scale-to-zero GPU node pool.
- Diagnosed cross-layer failures involving Kubernetes Service-link environment injection and a single-GPU RollingUpdate deadlock; implemented `enableServiceLinks: false` and a Recreate strategy.
- Built and load-tested a Horizontal Pod Autoscaler, correlating CPU metrics, scheduler events, GPU constraints, and cluster-autoscaler decisions to identify CPU as a weak signal for GPU inference.

ADD AFTER THE NEXT PHASE

- Instrumented vLLM with Google Managed Service for Prometheus and built Cloud Monitoring dashboards for queue depth, latency, throughput, and cache pressure.
- Implemented and validated custom-metric autoscaling based on inference queue pressure. **Do not use these bullets until completed.**

Portfolio rule: never claim “production-grade” from a single-node lab. Claim the real engineering: deployment, diagnosis, measurements, constraints, and a production-hardening plan.

A compact command map supports tomorrow's continuation

START AND VERIFY

```
kubectl scale deploy vllm --replicas=1
kubectl get pods -w
kubectl rollout status deployment/vllm --timeout=300s
kubectl logs deploy/vllm --tail=50
kubectl port-forward svc/vllm 8000:8000 &
curl -s http://localhost:8000/health
```

INSPECT RESOURCES AND AUTOSCALING

```
kubectl top pod
kubectl get hpa
kubectl describe hpa vllm
kubectl describe pod POD_NAME
kubectl get events --sort-by=.lastTimestamp
```

CREATE THE BASELINE CPU HPA

```
kubectl autoscale deployment vllm --cpu=70% --min=1 --max=3
```

The modern `--cpu=70%` syntax avoids the deprecated `--cpu-percent` flag. This HPA is useful for learning and baseline comparison, but the project evidence says it is not yet the preferred production signal.

INSPECT VLLM METRICS

```
curl -s http://localhost:8000/metrics | grep '^vllm:' | head
```

STOP SAFELY

```
kubectl delete hpa vllm
kubectl scale deploy vllm --replicas=0
kill $(jobs -p) 2>/dev/null
pkill -f "kubectl port-forward" 2>/dev/null
kubectl get pods
```

Order matters: delete an HPA with `minReplicas: 1` before scaling the Deployment to zero; otherwise the controller may restore a replica. Verify the GPU node eventually drains before considering the session cost-safe.

TOMORROW'S FIRST CHECKPOINT

- ☐ Confirm Deployment replica count is 0 and no GPU pod is running.
- ☐ Confirm GPU node pool has scaled down.
- ☐ Review the exact metrics emitted by the installed vLLM image.
- ☐ Begin Managed Prometheus collection only after the source endpoint is verified.

Speak precisely about the platform

HPA: Horizontal Pod Autoscaler; adjusts replica count from metrics.

Cluster autoscaler: adjusts node-pool size when pods cannot schedule for capacity.

Request: resource amount used for scheduling and utilization calculations.

Limit: runtime ceiling enforced for a resource; CPU can be throttled.

Millicore: one thousandth of a CPU core; 1000m equals one CPU.

Extended resource: discrete advertised resource such as `nvidia.com/gpu`.

Reconciliation: repeated controller action to move observed state toward desired state.

Pending: pod accepted but not yet scheduled or able to start.

ClusterIP: Service reachable only within the cluster network.

Port-forward: temporary local tunnel to a Kubernetes resource.

TTFT: time to first token, a key user-facing inference latency measure.

KV cache: GPU memory used to retain attention keys and values during generation.

PromQL: Prometheus query language.

PodMonitoring: Google managed-collection resource describing which pod metrics endpoint to scrape.

OFFICIAL REFERENCES

1. **vLLM Docker deployment:** <https://docs.vllm.ai/en/v0.16.0/deployment/docker/>
2. **Kubernetes Horizontal Pod Autoscaling:** <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>
3. **GKE cluster autoscaling:** <https://cloud.google.com/container-engine/docs/cluster-autoscaler>
4. **vLLM production metrics:** <https://docs.vllm.ai/en/v0.7.1/serving/metrics.html>
5. **Managed Prometheus collection example:** <https://cloud.google.com/stackdriver/docs/managed-prometheus/exporters/server/grpc>
6. **GKE custom metrics overview:** <https://cloud.google.com/blog/products/containers-kubernetes/gke-now-supports-custom-metrics-natively>
7. **GKE native Prometheus autoscaling update:** <https://cloud.google.com/blog/products/containers-kubernetes/native-support-for-prometheus-metrics-in-gke>

Preparation checklist: explain the architecture without notes; derive HPA math aloud; distinguish Pending from Running-not-Ready; tell the Recreate STAR story; name the observed metrics and constraints; describe next steps without claiming they are complete.